

Final Project :: RUNI 2025

Internet Tech / FullStack

DIY - Online Store



- Review one more time
 - Node: <https://medium.freecodecamp.org/the-definitive-node-js-handbook-69123>
 - Node API: <https://nodejs.org/docs/latest/api/>
 - NPM <https://vegibit.com/node-package-manager-tutorial/>
 - Package.json: <https://flaviocopes.com/package-json/>
 - Async/Await
 - <https://javascript.info/async-await>
 - https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Asynchronous/Async_await
 - <https://medium.com/@habibridho/javascript-es7-async-await-tutorial-64275c81ce2e>
 - JavaScript In the Browser
 - <https://developer.mozilla.org/en->

- [US/docs/Web/API/Document Object Model/Introduction](https://www.html5dog.com/docs/Web/API/Document_Object_Model/Introduction)
 - <https://www.html5dog.com/guides/javascript/intermediate/thedom/>
 - <https://www.html5dog.com/guides/javascript/intermediate/events/>
 - (optional) <https://reactjs.org/tutorial/tutorial.html>
 - (optional) <http://domenlightenment.com/>
- JSON <https://www.html5dog.com/guides/javascript/intermediate/json/>
- Local Storage <https://www.html5dog.com/guides/javascript/advanced/localstorage/>
- Errors and exceptions <https://www.html5dog.com/guides/javascript/advanced/errors/>
- HTTP methods (get/post/put/delete) <https://www.theserverside.com/blog/Coffee-Talk-Java-News-Stories-and-Opinions/HTTP-methods>
- Node.js modules: <https://www.javascripttutorial.net/nodejs-tutorial/nodejs-modules/>
-

Submit a zip file <yourID>_<firstName>_<lastName>_<partnerFullName>_<partnerId>_FP25.zip (e.g. '043462598_Ohad_Assulin_FP25.zip').

1. In this project you will build an online store
 - a. Support the following screens and features
 - i. Register screen - user **must** login in order to see any other page (excluding readme, products and the login/register screens)
 - ii. Login screen + "remember me" checkbox (if "remember me" cookie expires after 12 days, otherwise cookie expires after 30 minutes)
 1. One user should already exist: "admin"/"admin" (user/pass)
 - iii. Store screen:
 1. Presents the items which the store sells
 2. Textual prefix search (support searching products by their name or description)
 3. Add to cart button
 - a. if you are not logged in - clicking on this button should redirect you to the login screen
 - iv. Cart screen - shows products the user has added to the cart.
 1. Allow removing existing product from cart
 - v. Checkout screen (choose from the cart and "pay")
 - vi. Pay screen - fill your payment details and click pay. Upon payment, you show a "Thank you page" which indicates the payment was successful. (Note that it's a fake payment)
 - vii. My items screen - should show all the items you have bought.
 - viii. All screens come with a menu to enable easy navigation between screens. The menu should include a 'logout' button
 - ix. Admin screen that exposes the activity(login/logout/add-to-cart) of all users
 1. It should show a table whereas in each row, these are the columns: datetime, username, type of activity

2. Add a filter input box to see only the data of usernames that start with the filtered prefix (so if you type "abc" it will populate all the login and add-to-cart of all the users that their username starts with "abc")
3. Add a "manage products" section
 - a. Where the admin can add/remove new products to the store (product include title, description, picture (file or URL))
- x. Add additional pages with whatever functionality you want which make sense. Each page functionality must communicate with the server.. Those screens should not be similar to one another.
 1. **Minimum 4 additional pages.**
- xi. All the data about a specific user must be kept in file in the file system (however you want). It means that on server startup your application loads the existing data from the disk. The users and all of their data still exist and they can login into their accounts. Data that must be persisted:
 1. User details
 2. Cart
 3. Purchases
 4. Login activity
- xii. Defend from DOS attacks
- b. Add /readme.html route and describe there the following:
 - i. **The readme file must be done individually**
 - ii. Store name
 - iii. What are you selling?
 - iv. What additional page(s) did you add? How to operate it?
 - v. What was hard to do?
 - vi. Who is your partner? name and <id>. What did you do? What did your partner do?
 - vii. Specify all the different routes your app supports
 - viii. Explain how did you test it
- c. Add /llm.html route and specify all the code that LLM generated for you.
- d. Build test.js that test (intelligently and automatically) all the dynamic routes that your server supports. You only need to test the server-side routes. Use node-fetch to do that. test.js should print to the std out which tests it runs and which of them passed.
- e. Learn how package.json works and submit your work with the relevant package.json (Do not include the node_modules folder!)
- f. You submitted filename must comply:


```
<yourID>_<firstName>_<lastName>_<partnerFullName>_<partnerId>_FP25.zip
```
2. Additional requirements and allowances (Make sure you've implemented all those requirements):
 - a. You can use any npm package you need
 - b. You must work with **cookies** to implement user management
 - c. You must implement the server side of different screens (register, login, add to cart, products,...) in **different** node.js **modules**.
 - i. So the callback to the fetch()s of screen1.html should be reside in screen1-server.js module

- d. You must implement the **data persistence** in a **separate module** (the module that is in charge of saving/loading data from disk). You must call it `persist_module.js`
 - e. Utilize **async/await** for any async operation
 - f. Upon implementing routes, use must choose the **most suitable HTTP method** (get/post/put/delete)
 - g. You must **handle** any kind of **error**, exception and async error.
 - h. Use node fetch() to test your server
 - i. **(Optional) Challenge** - develop tests before the application itself.
 - 1. https://en.wikipedia.org/wiki/Test-driven_development
 - i. The user should be able to **customize** the view of the UI (you choose how, e.g. dark mode). You must persist/load the customization using the **localStorage**
 - j. You are allowed to use [React.js](#) (front end) and or [next.js](#) (backend)
 - k.
3. Grading
- a. Functionality / Instructions
 - b. Effort
 - c. Code (async-first, modules, efficiency)
 - d. User experience
 - e. Richness /Deepness
 - f. Security
 - g. Submission after September 2nd 23:59 will cause a penalty of 1 point per day
4. +10% of the students will be asked to explain their code/application over zoom

**Good
Luck**